



RS6x_7x mmWave Radar Reference Manual

Number of Pages:25

Document Version:1.2

Release Date:2026-08-31

Revision History

Version	Revision Description	Date
1.2	Added restrictions on using the hardware CFAR target acquisition interface in Section 3.4.3 Added the requirement to power on the DSP before accessing DataCube in Section 3.6	2026-08-13
1.1	Improved the introduction to the radar functional modules	2026-05-22
1.0	Initial release	2025-09-10

Table of Contents

Revision History	ii
Table of Contents	iii
List of Figures	v
List of Tables	vi
1 Overview	1
1.1 General Description	1
1.2 Component Introduction	1
1.2.1 Hardware Development Kit	1
1.2.2 Software Development Kit	1
1.3 Code and Tools	1
1.3.1 Radar Source Code Path	1
1.3.2 Related Tool Paths	1
1.4 Documentation	1
2 Development Environment	3
2.1 Overall Framework	3
2.2 Data Flow	3
3 Radar Functional Modules	5
3.1 Flow Control	5
3.1.1 mmw_ctrl Initialization	5
3.1.2 Registering the mmw_ctrl Callback Function	5
3.1.3 Opening/Closing DataCube	5
3.1.4 DSP Library	5
3.2 Frame Configuration	5
3.2.1 Configuring the Frame Type	6
3.2.2 Maximum Detection Range and Range Resolution	6
3.2.3 Maximum Unambiguous Velocity and Velocity Resolution	6
3.2.4 Chirp Accumulation Count	7
3.2.5 Frame Period and Frame Count	7
3.2.6 Starting Waveform Transmission	7
3.2.7 Hardware FFT Flow Control	7
3.2.7.1 Configuring Time-Domain Signal Output	8
3.2.7.2 Configuring Doppler FFT Enable	8
3.2.8 Frame Configuration Restrictions and Guidelines	8
3.3 Baseband Processing Configuration	9
3.3.1 Receive Antenna Gain and Transmit Antenna Attenuation	10
3.3.2 Window Functions	10
3.3.3 FFT Adaptive Gain	10
3.3.3.1 Adaptive Gain Restoration for a 1D Frame Structure	11
3.3.3.2 Adaptive Gain Restoration for a 2D Frame Structure	11
3.3.4 IF Filter (HPF)	12
3.3.5 Receive Antenna Calibration (Ant Align)	12
3.3.5.1 Enabling Antenna Calibration	12
3.3.5.2 Configuring the Calibration Matrix	12
3.3.6 Static Clutter Removal	12
3.3.6.1 Enabling Static Clutter Removal	13
3.3.6.2 Pausing Static Clutter Updates	13
3.3.6.3 Configuring the Static Clutter Convergence Rate	13
3.3.6.4 Resetting the Static Clutter Convergence Rate	13
3.3.6.5 Clearing the Static Clutter Estimate	13
3.3.7 Range-Doppler Phase Compensation (rd_decouple)	14

3.4 Hardware CFAR	14
3.4.1 CFAR Threshold and Static Target Removal	14
3.4.2 Doppler Local-Maximum Detection	15
3.4.3 Obtaining the CFAR Target List	15
3.5 Obtaining the Frame-Transmission Timestamp	15
3.6 DataCube Data Access	15
3.6.1 Reading Receive Antenna Data at a Specified Position	15
3.6.2 Reading All Range FFT Values at a Specified Receive Antenna/Doppler Index	16
3.6.3 Reading All Doppler FFT Values at a Specified Receive Antenna/Range Index	16
3.6.4 DataCube Data Access Restrictions	16
3.6.5 Configuring the Buffer for Range FFT Data from Multiple Intervals	17
4 Q & A	18
4.1 How Can the Radar Waveform Be Switched in Real Time?	18

List of Figures

Figure 2-1	Overall Radar Development Framework	3
Figure 2-2	MMW Processing Flow	4
Figure 3-1	Frame Structure Definition	6
Figure 3-2	Hardware FFT Data Flow	8
Figure 3-3	IF Signal Processing Flow	10
Figure 3-4	Adaptive Gain Restoration for a 1D Frame Structure	11
Figure 3-5	Adaptive Gain Restoration for a 2D Frame Structure	11
Figure 3-6	Static Clutter Control Logic	13
Figure 3-7	Checkpoints for Software Access to DataCube	16

List of Tables

Table 3-1	Typical Configurations Corresponding to Memory Usage	9
Table 3-2	Typical Configurations Corresponding to Memory Usage (2x Compression)	9

1 Overview

This document primarily describes how to use the radar module of the RS6-series mmWave radar sensor chips and provides answers to frequently asked questions. It is intended for personnel developing radar-chip application solutions.

1.1 General Description

Possumic RS6x_7x-series chips are 60 GHz ultra-low-power mmWave radar sensor chips. Each chip integrates mmWave RF and BB subsystems and hardware acceleration modules for radar computations such as FFT and CFAR, enabling moving-target sensing and positioning. With rich peripheral interfaces, the chips are widely applicable to smart homes, smart appliances, intelligent security, smart lighting, and other applications. Their ultra-low-power design also provides longer battery life for battery-powered products.

This document primarily introduces the radar functions of the RS6x_7x development package to help developers build radar products more efficiently.

1.2 Component Introduction

1.2.1 Hardware Development Kit

1. MRS6130_P1812/MRS6130_P1806/MRS6240_P2512/MRS6241_P2828_M62/MRS6241_P2840_M81: mmWave radar modules;
2. MRS6130-AIP-DEV-V1/MRS6240-AIP-DEV-V1/MRSx241-DEV-V1: development boards carrying the mmWave radar modules;
3. USB Type-C cable: cable used to connect a PC to the development board.

1.2.2 Software Development Kit

1. SDK: code and build environment for software development;
2. RadarAnalysisTool: PC-based radar GUI tool for radar performance evaluation and function demonstration;
3. RadarDebugTool: PC-based radar visualization tool for radar algorithm development.

1.3 Code and Tools

1.3.1 Radar Source Code Path

- MMW API: [/02_SDK/psdf_sdk/subsys/mmwave/mmwave_ctrl/inc/mmwave_ctrl.h](#)
- Sample: [/02_SDK/psdf_sdk/project/mmwave/](#)

1.3.2 Related Tool Paths

- Serial port driver: [Software Kit/03 Tool/USB Serial Driver/](#)
- burn tool: [Software Kit/03 Tool/BurnTool](#)

1.4 Documentation

- SDK Reference Manual: [Software Kit/01 Document/RS6x_7x_SDK_Reference_Manual.pdf](#)
- Evaluation Quick Start Guide: [Software Kit/01 Document/RS6x_7x_mmWave_Radar_Evaluation_Quick_Start_Guide.pdf](#)
- Development Quick Start Guide: [Software Kit/01 Document/RS6x_7x_mmWave_Radar_Development_Quick_Start_Guide.pdf](#)

- Point Cloud Algorithm Reference Manual:
[Software Kit/01 Document/RS6x 7x mmWave Radar Point Cloud Process Reference Manual.pdf](#)
- Radar Sample Project Reference Manual:
[Software Kit/01 Document/RS6x 7x mmWave Radar Sample Reference Manual.pdf](#)
- Radar Reference Manual:
[Software Kit/01 Document/RS6x 7x mmWave Radar Reference Manual.pdf](#)
- Module User Manual:
[Hardware Kit/01 Document/MRS6130-P1812 mmWave Radar Module Datasheet.pdf](#)
- Module User Manual:
[Hardware Kit/01 Document/MRS6130-P1806 mmWave Radar Module Datasheet.pdf](#)
- Module User Manual:
[Hardware Kit/01 Document/MRS6240-P2512 mmWave Radar Module Datasheet.pdf](#)

2 Development Environment

This chapter describes how to develop applications using radar functions in the SDK environment, including the overall framework, data flow, build configuration, and typical examples. This chapter covers only radar-related content. For SDK download and basic usage, see the SDK Reference Manual listed in Section 1.4, Documentation.

2.1 Overall Framework

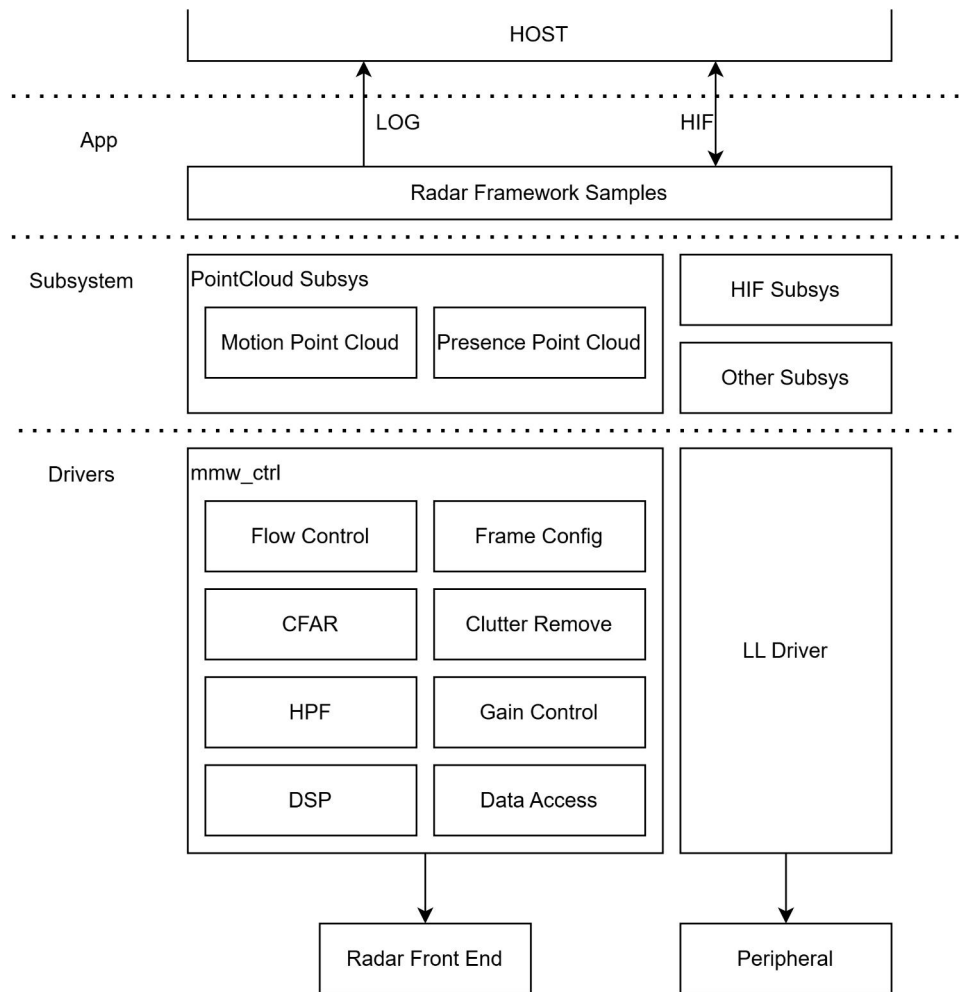


Figure 2-1 Overall Radar Development Framework

2.2 Data Flow

As shown in Figure 2-2, the standard radar data flow is as follows:

1. Open and initialize the radar module through the `mmw_ctrl_open` interface;
2. Use the configuration interfaces in Chapter 3 to configure the radar frame-structure parameters and baseband processing settings;
3. Register the radar data-processing callback function and data callback type through the `mmw_data_callback` interface;
4. Start radar operation through the `mmw_ctrl_start` interface;
5. During operation, the radar module transmits frames and triggers the callback function registered in Step 3 after completing internal data processing;
6. Read the required data in the callback function and process it at the application level;
7. Repeat Step 5. The radar stops when the number of transmitted frames reaches the user-configured count or when `mmw_ctrl_stop` is called.

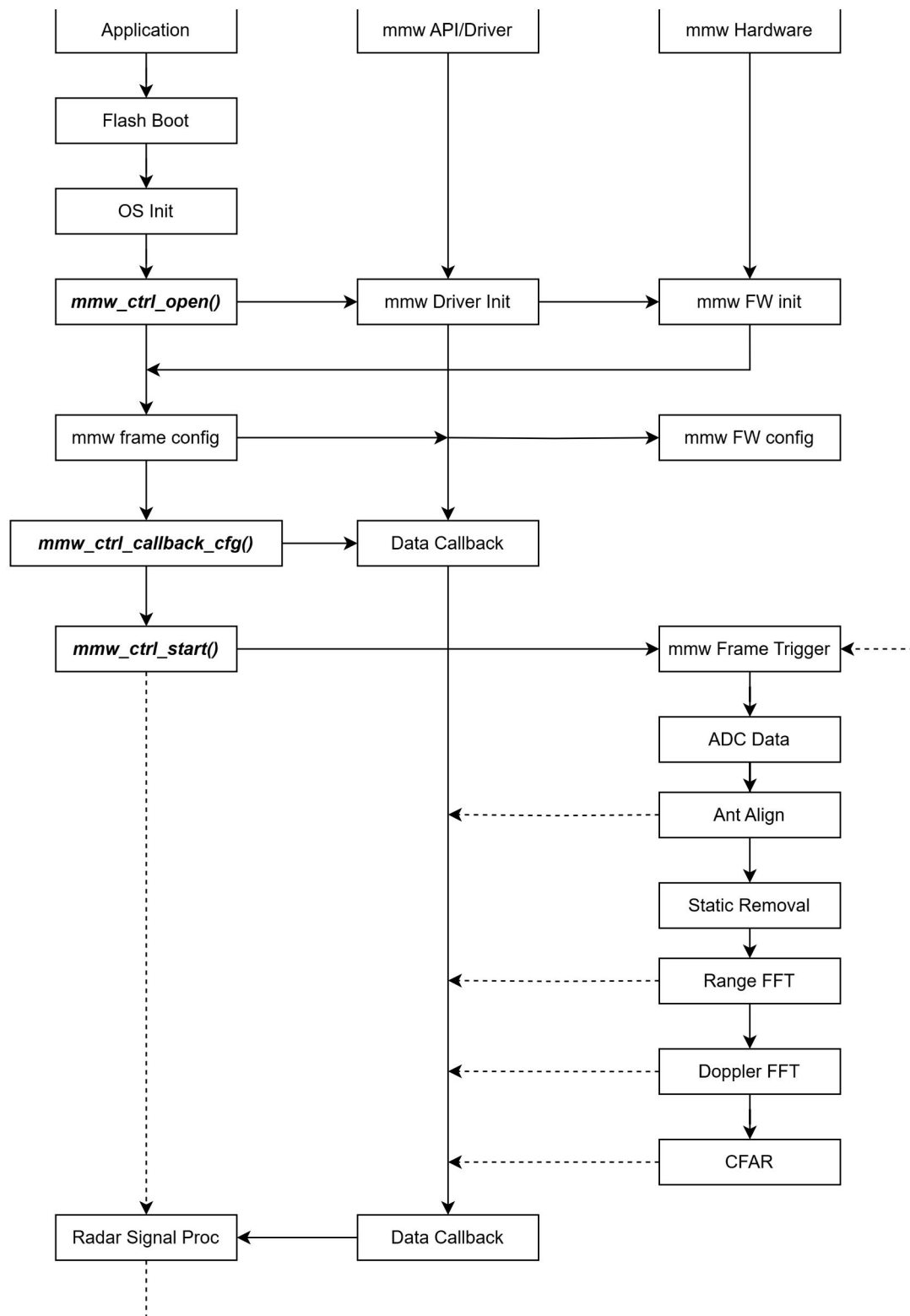


Figure 2-2 MMW Processing Flow

3 Radar Functional Modules

This chapter describes the commonly used radar-module functions and their corresponding APIs in detail.

The `mmw_ctrl` module bridges the radar hardware and the application. Its main functions include:

1. Configuration interfaces between the application layer and the Radar Front End and baseband processing;
2. Interfaces through which the application layer obtains baseband processing results such as DataCube data, CFAR target lists, and the noise floor;
3. Interrupt and callback registration and deregistration.

3.1 Flow Control

`mmw_ctrl` flow control mainly includes `mmw_ctrl` initialization, startup, interrupt registration, and shutdown.

3.1.1 `mmw_ctrl` Initialization

Before performing any `mmw_ctrl`-related operation, call **`mmw_ctrl_open(bool ref_msi, bool pll_lc, bool force_calib)`** to initialize the radar controller and select the reference clock. The reference clock may affect timing accuracy and power consumption.

It is recommended to start the `mmw_ctrl` component only by using the fixed call **`mmw_ctrl_open(true, false, true)`**. After this function is called, the radar firmware requires approximately 1 second to complete `mmw_ctrl` initialization.

3.1.2 Registering the `mmw_ctrl` Callback Function

Use **`mmw_ctrl_callback_cfg()`** to register the radar callback function. The callback is executed after the hardware completes the corresponding operation, according to the registered interrupt type (1D-FFT/2D-FFT).

3.1.3 Opening/Closing DataCube

Use **`mmw_motion_cube_access_open()`** before reading DataCube data to notify the hardware to perform the relevant preparation, thereby improving DataCube access efficiency. Note: Do not call this function before obtaining the hardware CFAR target list, or the obtained list will be incorrect.

Use **`mmw_motion_cube_access_close()`** after DataCube data access is complete to notify the hardware to perform the relevant deinitialization.

3.1.4 DSP Library

Use **`mmw_dsp_poweron()`** to power on the on-chip DSP so that subsequent DSP-related functions (`mmw_dsp_xxx`) can operate correctly. The DSP power state is reset at the beginning of every frame; therefore, developers must call **`mmw_dsp_poweron()`** first in each callback function to ensure that subsequent radar algorithm-library functions operate correctly. Some `mmw_ctrl` interfaces use the DSP library, so calling **`mmw_dsp_poweron()`** at the beginning of the callback function is strongly recommended.

3.2 Frame Configuration

The waveform structure in `mmw_ctrl` is shown below:

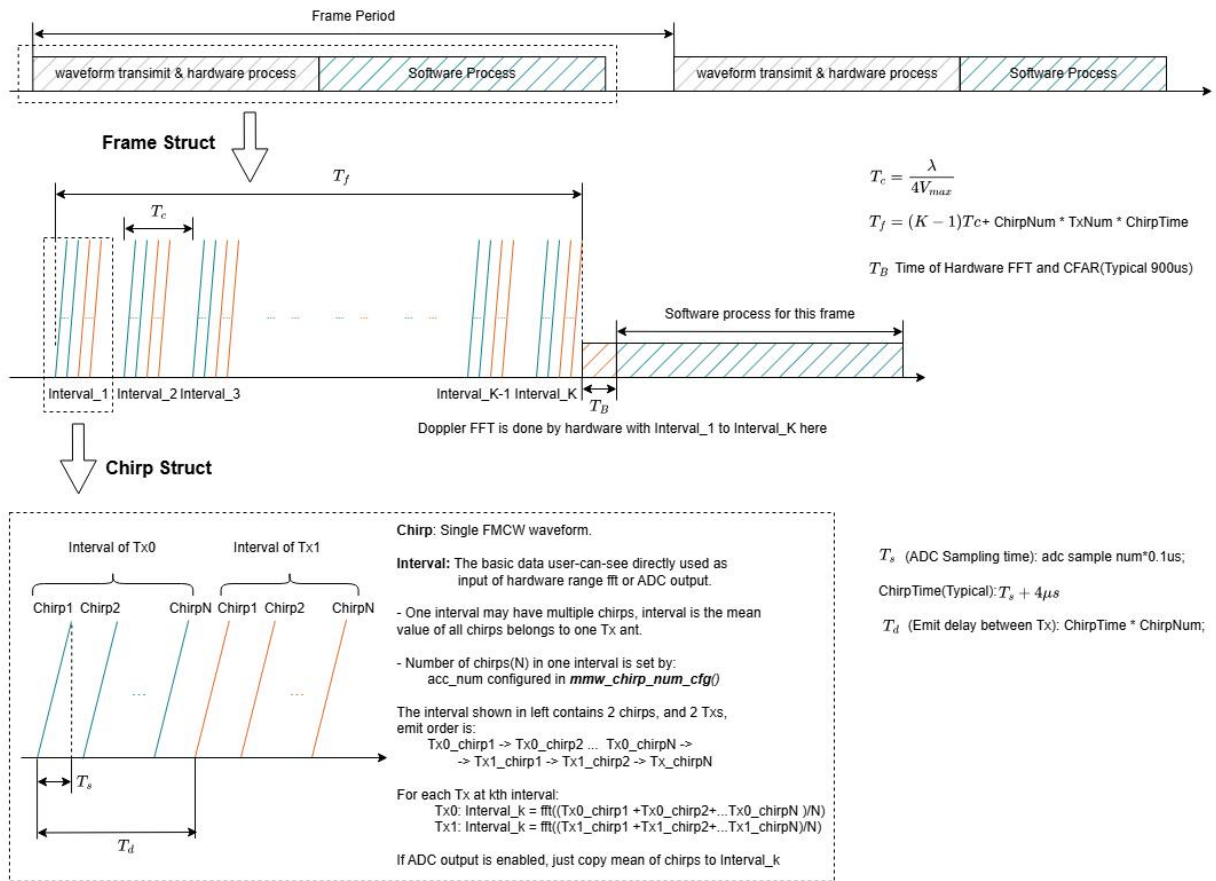


Figure 3-1 Frame Structure Definition

The frame-structure-related APIs and their correspondence to the parameters in Figure 3-1 are listed below.

3.2.1 Configuring the Frame Type

`mmw_mode_cfg(uint8_t txrx_mode, uint8_t work_mode)`

Configure the radar MIMO mode and hardware FFT behavior. For a multiple-transmit, multiple-receive (MIMO) system, the radar front end currently supports only TD-MIMO waveform transmission. Figure 3-1 shows the RS6240 frame structure when `txrx_mode` is set to 2T4R and `work_mode` is set to `MMW_WORK_MODE_2DFFT`. When `work_mode` is set to `MMW_WORK_MODE_1DFFT`, the difference from 2D frame mode is that each Frame transmits only one Interval.

3.2.2 Maximum Detection Range and Range Resolution

`mmw_range_cfg(uint32_t range_mm, uint32_t resol_mm)`

Configure the maximum detection range and range resolution. This function determines the number of ADC samples in each Chirp:

$$\text{ADC_Sample_Num} = \text{floor}(\text{range_mm}/\text{resol_mm})$$

The SDK currently supports only a 10 MHz ADC sampling rate (complex sampling). The sampling time is:

$$\text{Sampling_time} = \text{ADC_Sample_Num} * 0.1 \mu s$$

This corresponds to the parameter in Figure 3-1. T_s

This function also determines the number of range FFT points, calculated as follows:

$$\text{Range_FFT_Length} = \text{power}(2, \text{ceil}(\text{range_mm}/\text{resol_mm}));$$

The configurable ADC sample-count range is 32 to 1024 for 1T3R/2T4R with FFT adaptive gain disabled, or 32 to 512 for 2T4R with FFT adaptive gain enabled.

3.2.3 Maximum Unambiguous Velocity and Velocity Resolution

`mmw_velocity_cfg(uint32_t velocity_mm, uint32_t veloc_resol)`

Configure the maximum unambiguous velocity (velocity_mm) and velocity resolution (veloc_resol). This function is valid only when the frame type is configured as a 2D frame. The effective radar velocity-measurement range is [-velocity_mm, velocity_mm). This function determines the number K of Intervals in Figure 3-1: $K = \text{floor}(2 * \text{velocity_mm} / \text{veloc_resol})$. It also determines the number of Doppler FFT points, calculated as $2 * \text{velocity_mm} / \text{veloc_resol}$ and rounded up to a power of 2.

In addition, the maximum unambiguous velocity determines the Interval spacing $T_c = \frac{\lambda}{4V_{\max}}$, and the

velocity resolution determines the Frame transmission time: $T_f = \frac{\lambda}{2V_{\text{resol}}}$ (approximate value) or

$$T_f = T_c \times (K - 1)$$

The configurable range of the number K of Intervals is 2 to 512.

3.2.4 Chirp Accumulation Count

mmw_chirp_num_cfg(uint32_t num)

Configure the Chirp accumulation count, which is the number N of Chirps in one Interval in Figure 3-1. Baseband processing performs complex averaging on all Chirps within an Interval and uses the result as the Interval sample for the range FFT. Configuring more Chirps per Interval gives baseband processing a higher signal-to-noise ratio. This parameter can be configured only to a power of 2.

3.2.5 Frame Period and Frame Count

mmw_frame_cfg(uint32_t period_ms, uint32_t frame_num)

Configure the Frame Period and transmitted frame count shown in Figure 3-1, and **apply the waveform parameters configured through the other APIs** in this chapter. A transmitted frame count of 0 means that frames are transmitted continuously at the configured frame period; a nonzero value means that RF-signal transmission stops after the configured number of frames has been transmitted.

3.2.6 Starting Waveform Transmission

Call **mmw_ctrl_start()** to start the RF front end. **Before calling this function, complete all mmw_ctrl-related settings through mmw_ctrl frame configuration and baseband processing configuration.**

3.2.7 Hardware FFT Flow Control

mmw_ctrl supports the following configurations:

1. Skip the range FFT stage during hardware processing;
2. Skip the Doppler FFT stage during hardware processing;
3. Buffer range FFT data from multiple Intervals after the hardware performs the 1D-FFT.

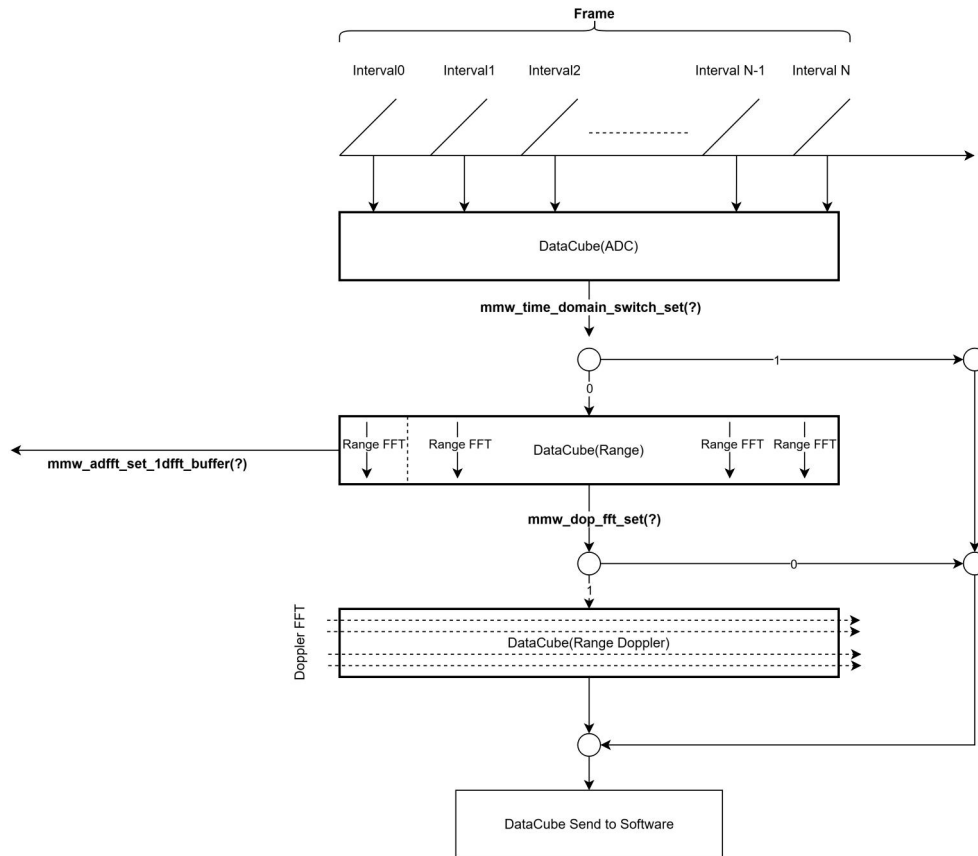


Figure 3-2 Hardware FFT Data Flow

3.2.7.1 Configuring Time-Domain Signal Output

mmw_time_domain_switch_set(uint8_t enable)

Configure whether time-domain signals are output. If enable is set to 1, the DataCube read by software contains the data before the range FFT is performed for each Interval, and subsequent range FFT and Doppler FFT calculations are no longer performed.

Note: When using this function, the number of ADC samples and the number of Intervals in the frame configuration must both be powers of 2. This function must be called between `mmw_ctrl_open()` and `mmw_ctrl_start()`. Time-domain signal output is disabled by default.

Note: The output ADC signal has undergone the static clutter removal and Chirp accumulation processing described in Section 3.3.

3.2.7.2 Configuring Doppler FFT Enable

mmw_dop_fft_set(uint8_t on_off)

Configure whether the Doppler FFT is enabled. When set to 0, the radar hardware does not perform the Doppler FFT, and the DataCube read by software contains the range FFT data for each Interval.

Note: When using this function, the number of ADC samples and the number of Intervals in the frame configuration must both be powers of 2. This function must be called between `mmw_ctrl_open()` and `mmw_ctrl_start()`. The Doppler FFT is enabled by default.

Note: When the number of Doppler FFT points is 2, only disabling the Doppler FFT is supported.

3.2.8 Frame Configuration Restrictions and Guidelines

1. Unless there are special circumstances or special requirements for subsequent digital signal processing, it is generally recommended to configure the maximum detection range and range resolution so that the number of ADC samples is exactly a power of 2 and equals the number of range FFT points. Recommended range FFT point counts are 128, 256, and 512. Point-cloud computation load increases as the range FFT point count increases;
2. Unless there are special circumstances or special requirements for subsequent digital signal processing, it is generally recommended to configure the maximum unambiguous velocity and velocity resolution so that

the number of Intervals is exactly a power of 2 and equals the number of Doppler FFT points. Point-cloud computation load increases as the Doppler FFT point count increases;

3. It is recommended to configure the maximum detection range to greater than 5 m. By default, the last 20% of the range spectrum is unavailable. If the farthest target to be detected is at range R, adjust the range FFT point count and range resolution so that the maximum detection range MaxRange and R satisfy:

$$R \in [0.67 \text{MaxRange}, 0.8 \text{MaxRange}],$$

The purpose is to maximize spectrum utilization. If $R < 5$ m, the maximum detection range is not constrained by the preceding equation; it is sufficient to set $\text{MaxRange} > 5$ m.

4. Each IQ data item in DataCube is represented by 4 bytes. The space occupied by the entire DataCube is calculated as follows:

$$\text{Range_FFT_Length} * \text{Doppler_FFT_Length} * \text{Tx_num} * \text{Rx_num} * 4(\text{Bytes})$$

For RS624x, the maximum storable DataCube size is 256 KB; for RS6130, it is 128 KB. When the SRAM corresponding to the configured DataCube size exceeds 256 KB, the hardware automatically enables lossy data compression, compressing data during storage and automatically decompressing it during reading. Up to 2x compression is supported, so the maximum configurable DataCube size is 512 KB for RS624x and 256 KB for RS6130. However, 2x compression reduces the radar dynamic range and may raise the quantization noise on range bins containing strong signals.

Typical configurations that fully occupy the DataCube storage space are as follows:

Table 3-1 Typical Configurations Corresponding to Memory Usage

Chip Model	Tx/Rx Antenna Selection	Range FFT Points	Velocity FFT Points	Memory Usage
RS624x	2T4R	256/128/64	32/64/128	$2 * 4 * 256 * 32 * 4 = 256\text{K}$
RS6130	1T3R	256/128/64	32/64/128	$1 * 3 * 256 * 32 * 4 = 96\text{K}$

Table 3-2 Typical Configurations Corresponding to Memory Usage (2x Compression)

Chip Model	Tx/Rx Antenna Selection	Range FFT Points	Velocity FFT Points	Memory Usage
RS624x	2T4R	256/128/64	64/128/256	$2 * 4 * 256 * 64 * 2 = 256\text{K}$
RS6130	1T3R	256/128/64	64/128/256	$1 * 3 * 256 * 64 * 2 = 96\text{K}$

5. Only after `mmw_frame_cfg()` is called can waveform parameters be obtained through the get functions provided by the SDK, such as `mmw_range_get()`;

6. The signal-to-noise ratio and the detection capability for weakly reflecting targets can be improved in the following two ways:

- (1) Preferably increase the Chirp accumulation count to improve the signal-to-noise ratio. This does not require modifying the waveform parameters related to point-cloud computation and has little effect on point-cloud density;
- (2) Increase the range FFT and Doppler FFT point counts.

7. When the number of Doppler FFT points is 2, only disabling the Doppler FFT is supported.

3.3 Baseband Processing Configuration

Baseband processing configuration covers the complete process from IF signal input to generation of the CFAR target list, as shown in Figure 3-3:

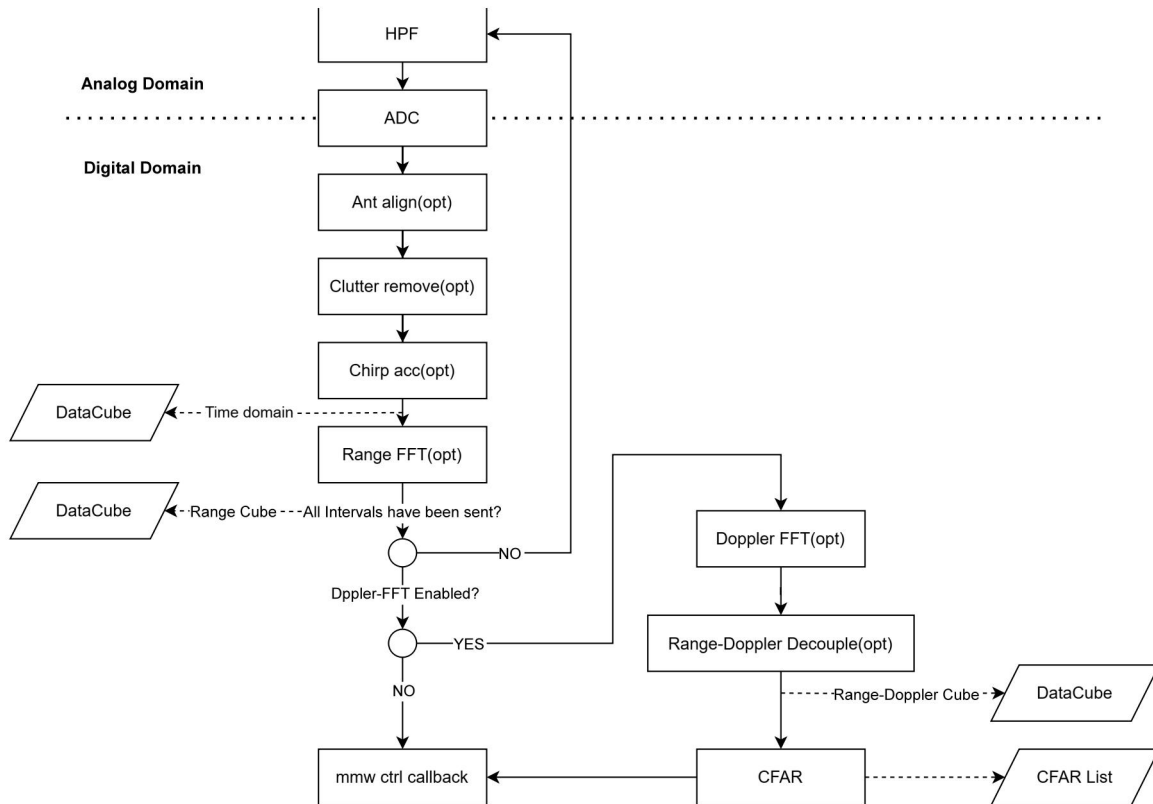


Figure 3-3 IF Signal Processing Flow

The baseband processing configurations described in this chapter must be performed after ***mmw_frame_cfg(uint32_t period_ms, uint32_t frame_num)*** is called.

3.3.1 Receive Antenna Gain and Transmit Antenna Attenuation

mmw_rx_ana_gain_cfg(uint8_t rx_ant_id, uint8_t gain)

Configure the receive gain (power) of the receive antennas. The default is 51 dB, and five levels are configurable: 27/33/39/45/51 dB. For example, if the normalized power measured for a corner reflector is -10 dB when the receive antenna gain is set to 51 dB, the corresponding corner-reflector power is -16 dB when the receive antenna gain is set to 45 dB;

mmw_tx_rf_att_set(int att_db)

Configure Tx transmit-power attenuation. The default attenuation is 0 dB, and the configurable attenuation range is 0 to 8 dB. All transmit antennas share this attenuation value. If the normalized power measured for a corner reflector is -10 dB when transmit-power attenuation is 0 dB, the corresponding corner-reflector power is -16 dB when transmit-power attenuation is 6 dB. This function is useful in scenarios where transmit power must be controlled.

3.3.2 Window Functions

mmw_win_type_set(uint8_t calType, uint8_t win_type)

Set the specified window functions for the range FFT and Doppler FFT calculated by hardware. Currently, only rectangular or Hann windows are supported. The range FFT and Doppler FFT both use Hann windows by default.

3.3.3 FFT Adaptive Gain

mmw_fft_autogain_set(uint8_t enable)

Enable FFT adaptive gain. When a strong target appears in the environment, the hardware FFT automatically adjusts the calculation gain so that the strong target signal in DataCube does not overflow while the quantization noise of weak targets does not deteriorate.

3.3.3.1 Adaptive Gain Restoration for a 1D Frame Structure

In a 1D frame structure, the range FFT data for each Interval has an adaptive FFT gain for each different transmit antenna. This gain is obtained through `mmw_fft_autogain_interval(uint32_t interval_idx, uint8_t tx_idx)`.

The gain restoration method is as follows:

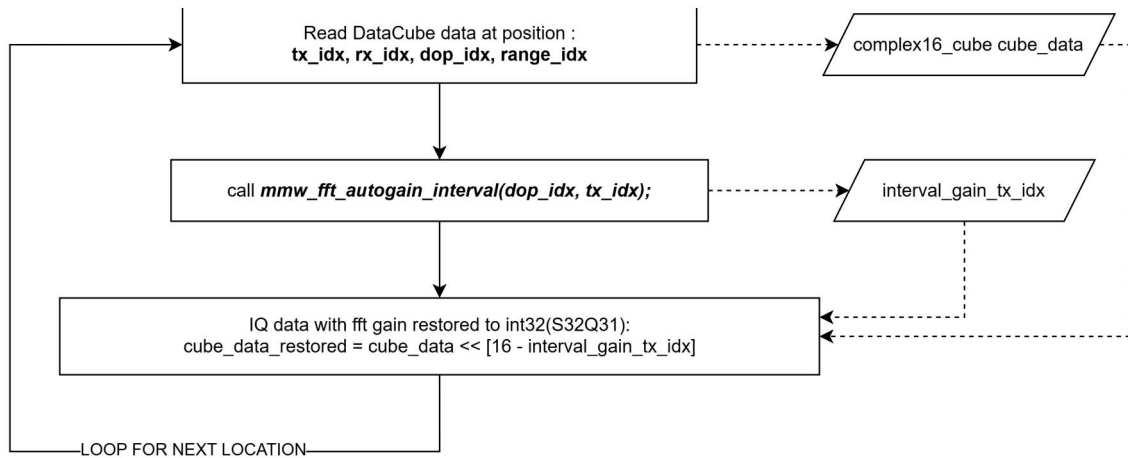


Figure 3-4 Adaptive Gain Restoration for a 1D Frame Structure

3.3.3.2 Adaptive Gain Restoration for a 2D Frame Structure

For a 2D frame structure, there are two cases: with and without Doppler FFT execution. When the Doppler FFT is not executed, the adaptive gain restoration method is the same as that shown in Figure 3-4.

When the Doppler FFT is executed, the adaptive gain in each DataCube consists of two parts:

1. The base gain of the Tx corresponding to the DataCube data, obtained through `mmw_fft_autogain_base(uint8_t tx_idx)`;
2. The individual gain corresponding to the specified range bin and Tx of the DataCube data, obtained through `mmw_fft_autogain_range(uint32_t range_idx, uint8_t tx_idx)`.

When the Doppler FFT is enabled, the adaptive gain restoration method is shown in Figure 3-5.

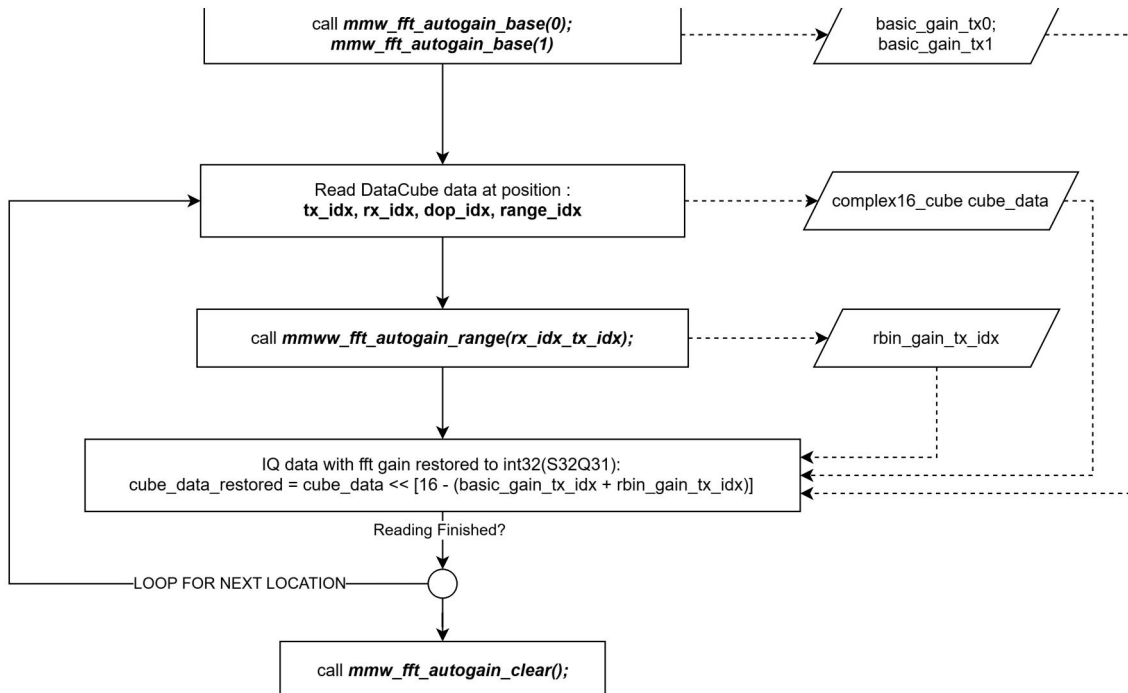


Figure 3-5 Adaptive Gain Restoration for a 2D Frame Structure

Note: In a 2D frame structure, after all adaptive gains have been read, `mmw_fft_autogain_clear()` must be called manually to clear the adaptive gains before waveform transmission begins for the next frame.

3.3.4 IF Filter (HPF)

mmw_ana_hpf_bw_cfg(uint8_t rx_ant_id, uint8_t bw)

Configure the bandwidth of the high-pass filter applied to the IF signal. The high-pass filter suppresses energy leaked from the direct wave into nearby targets. The IF high-pass filter bandwidth can be configured for each receive antenna or for all receive antennas. A wider high-pass filter bandwidth provides stronger suppression of low-frequency signals (nearby targets), preventing them from exceeding the ADC range.

In addition, target energy near the detection-range limit may couple to the nearest range, and energy from targets at the nearest range may couple to the detection-range limit (range aliasing). This phenomenon becomes more severe when the detection range is very short. Using a wider IF filter bandwidth provides better suppression of these aliased signals and prevents false detections caused by coupled distant signals.

The default high-pass filter bandwidth is 0.5 MHz.

3.3.5 Receive Antenna Calibration (Ant Align)

3.3.5.1 Enabling Antenna Calibration

mmw_set_ant_align(uint8_t mode)

Enable the hardware calibration function. Calibration is performed in hardware after the ADC data is acquired.

For each sample, calibration left-multiplies the vector formed by the four receive antennas by a calibration matrix:

$$\begin{bmatrix} rx0 \\ rx1 \\ rx2 \\ rx3 \end{bmatrix}_{aligned} = \begin{bmatrix} Para_{0,0} & Para_{0,1} & Para_{0,2} & Para_{0,3} \\ Para_{1,0} & Para_{1,1} & Para_{1,2} & Para_{1,3} \\ Para_{2,0} & Para_{2,1} & Para_{2,2} & Para_{2,3} \\ Para_{3,0} & Para_{3,1} & Para_{3,2} & Para_{3,3} \end{bmatrix} \begin{bmatrix} rx0 \\ rx1 \\ rx2 \\ rx3 \end{bmatrix}$$

In the preceding equation, [rx0,rx1,rx2,rx3] is the IQ data for up to four receive antennas. When RS6130 is used, rx3 is always 0. In a TD-MIMO system, the same calibration matrix is used for every Tx. In practice, different Tx antennas may require different calibration matrices. Therefore, when RS624x is used, it is recommended not to enable receive antenna calibration, but instead to calibrate the DataCube data in software.

3.3.5.2 Configuring the Calibration Matrix

mmw_set_ant_align_data(const complex16_reim calib_mat[16])

Configure the calibration matrix described in Section 3.3.5.1. calib_mat is a one-dimensional array with a fixed length of 16, arranged as follows:

$$Para_{0,0} \dots Para_{0,3}, Para_{1,0} \dots Para_{1,3}, Para_{2,0} \dots Para_{2,3}, Para_{3,0} \dots Para_{3,3}$$

The real or imaginary part of each calibration value is an S13Q11 integer. The default calibration matrix is an identity matrix.

3.3.6 Static Clutter Removal

Static clutter removal eliminates the energy of zero-velocity targets on the range-Doppler spectrum for the Rx antennas corresponding to all Tx antennas, bringing it close to the noise floor. Static clutter removal is recommended only for 2D frame structures.

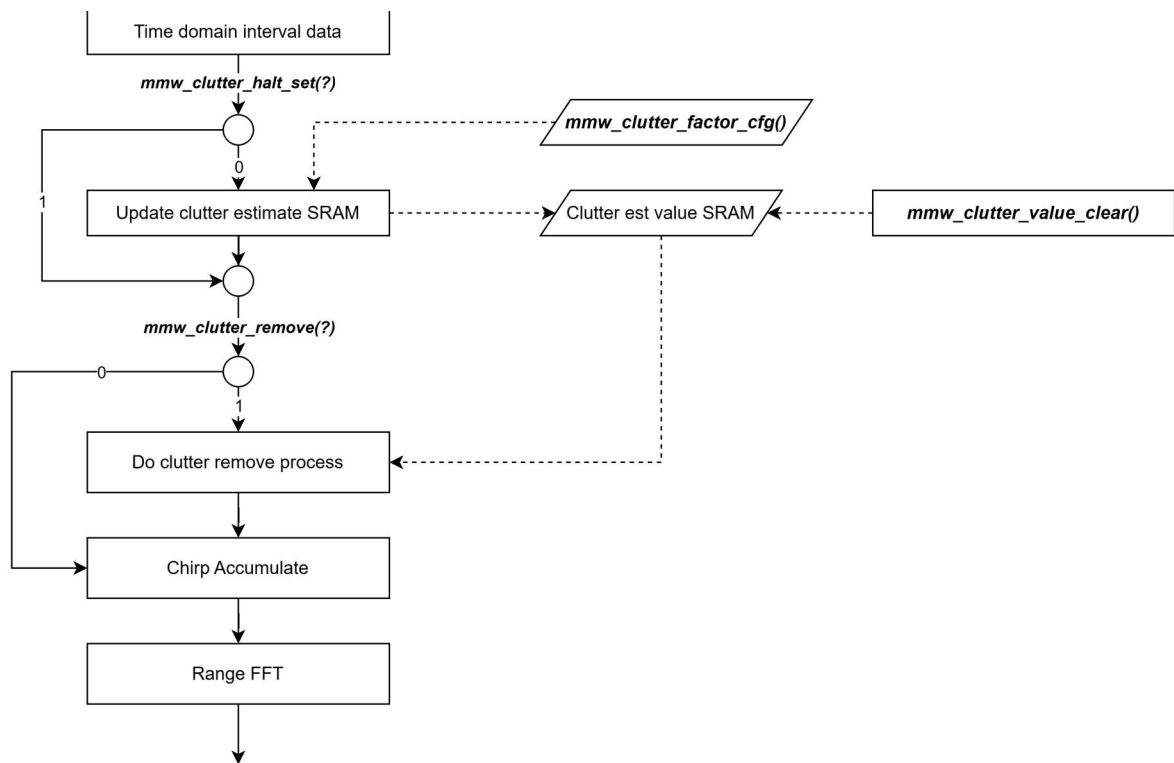


Figure 3-6 Static Clutter Control Logic

3.3.6.1 Enabling Static Clutter Removal

mmw_clutter_remove(uint8_t clutter_mode)

Configure static clutter removal. When this function is enabled, the hardware stores the estimated time-domain energy of static targets for each Frame in a section of SRAM and subtracts it before each range FFT. As a result, the energy of stationary targets is suppressed in the final DataCube.

Note: Disable static clutter removal if the radar is installed on a moving platform.

3.3.6.2 Pausing Static Clutter Updates

mmw_clutter_halt_set(uint8_t halt)

Configure whether the hardware is allowed to update the time-domain static-target energy. When halt is set to 1, the hardware keeps the SRAM data containing the estimated static clutter energy unchanged and still subtracts the static clutter (controlled by *mmw_clutter_remove()*). This feature is often used to detect slowly moving targets in the environment, which might otherwise be subtracted as static clutter. If static clutter updates are paused before such targets appear, they will not be subtracted as static clutter.

3.3.6.3 Configuring the Static Clutter Convergence Rate

mmw_clutter_factor_cfg(uint8_t clutter_factor)

Configure the time constant for static clutter estimation. A larger clutter_factor produces slower static clutter convergence but greater static clutter suppression. When a static target impulse occurs, the number of frames required for static clutter convergence is approximately $2 \ll \text{clutter_factor}$. It is recommended to configure clutter_factor to no more than 7.

3.3.6.4 Resetting the Static Clutter Convergence Rate

mmw_clutter_reset_fast_convergence_flag()

Reset the static clutter convergence rate. When the waveform configuration or environment changes abruptly, the static clutter low-pass filter weakens the ability of the static clutter estimate to track the environment, causing false detections. This function resets fast convergence for static clutter, allowing rapid tracking of changes in environmental static clutter and accelerating convergence.

3.3.6.5 Clearing the Static Clutter Estimate

mmw_clutter_value_clear()

Use this function to reset all values in the SRAM that stores the hardware-estimated static clutter to 0.

3.3.7 Range-Doppler Phase Compensation (rd_decouple)

mmw_set_rd_decouple(uint8_t enable)

Enable the hardware to compensate, during DataCube calculation, for the phase difference caused by different transmission times of the two Tx antennas. This function is supported only on RS624x and is effective only when 2D-FFT calculation is enabled through ***mmw_dop_fft_set(uint8_t on_off)***.

The transmission times of the two Tx antennas correspond to the values in Figure 3-1. For a target with Doppler index k , the additional phase difference on the receive antennas corresponding to Tx1 relative to Tx0 is: T_d T_d

$$\varphi = 2\pi \frac{T_d}{KT_c} \times k$$

In the preceding equation, K is the number of Doppler FFT points, k is the target Doppler index (0 indicates stationary and >0 indicates movement away from the radar), and the remaining terms are defined with reference to Figure 3-1. T_d T_c

If range-Doppler phase compensation is enabled, the hardware calculates the phase according to the Doppler index while computing DataCube and subtracts it from the four Rx antennas corresponding to Tx1 for each IQ data item, eliminating the phase offset caused by the Tx transmission-time difference. φ φ

Observe the following when using this function:

1. If the target velocity exceeds the maximum detectable velocity, causing velocity ambiguity, the phase difference automatically calculated by hardware is no longer accurate. However, because the relevant timing difference is much smaller than the interval, range-Doppler phase compensation can be disabled when angle-estimation accuracy requirements are not high; φ T_d T_c

2. If the developer wishes to use data on which range-Doppler detection has not been performed for subsequent processing, the U32Q20 value obtained through ***mmw_get_dec_fcw()*** can be used to perform

range-Doppler phase compensation on DataCube in subsequent software processing. $\frac{T_d}{KT_c}$

3.4 Hardware CFAR

After DataCube is completed, the hardware can execute hardware CFAR, corresponding to the stage shown in Figure 3-1. Hardware CFAR calculates the ratio between the signal magnitude and the noise-floor magnitude at each range and Doppler index in DataCube. If the ratio exceeds the detection threshold configured by software, the target is added to the target list. T_B

3.4.1 CFAR Threshold and Static Target Removal

mmw_cfar_cfg(bool static_remove, uint32_t snr)

Configure the hardware CFAR detection threshold and static-target bypass function. The relationship between the signal-to-noise ratio (snr) configured through this function and the signal-to-noise ratio in dB (snr_db) is:

$$snr = \text{floor}(\text{power}(10, snr_db/20) * 8)$$

In general, ensure that snr_db is within the range of 10 dB to 30 dB.

Hardware CFAR provides a static target removal function (static_remove). If static_remove is set to 1, hardware CFAR does not add targets with Doppler indices $[N/2-1, N/2, N/2+1]$ (where N is the number of Doppler FFT points) to the CFAR target list. This is useful for applications that need to filter out relatively stationary targets. It is recommended to enable static_remove when the radar is stationary and stationary targets are not of interest.

Important characteristics of hardware CFAR:

1. The SNR calculation accuracy of the hardware CFAR module is relatively low, and it may output point clouds whose actual signal-to-noise ratio is below the configured threshold. It is recommended to configure a relatively low hardware CFAR threshold and recalculate the signal-to-noise ratio in software for secondary filtering during subsequent processing;
2. The maximum signal-to-noise ratio output by the hardware CFAR module is 32 dB;

3. Hardware CFAR detects targets from far to near and can store up to 512 targets. Configuring a CFAR threshold that is too low or disabling static target removal may saturate hardware CFAR and cause nearby targets to be lost. After the hardware CFAR list is saturated, detection continues, and the first target output in the CFAR list is continuously replaced.

3.4.2 Doppler Local-Maximum Detection

mmw_cfar_stationary(bool enable)

Configure whether local-maximum detection must be performed in the Doppler dimension after the hardware CFAR list is read. Hardware CFAR local-maximum detection only removes targets that are not local maxima in the Doppler dimension from the hardware CFAR output list.

3.4.3 Obtaining the CFAR Target List

mmw_cfar_result(CFAR_Result *res, CFAR_Info *info, uint32_t *cfar_num, uint32_t buf_num)

Read the CFAR list. Provided the CFAR list is not read out of bounds, this function reads at most the first `buf_num` targets in the CFAR list from near to far and returns the actual number read through `cfar_num`.

Restrictions:

1. Before calling the interface for obtaining the CFAR target list, call ***mmw_dsp_poweron()*** to wake up the DSP; otherwise, incorrect CFAR results will be obtained;
2. Do not call ***mmw_motion_cube_access_open()*** before calling the interface for obtaining the CFAR target list. If it has already been called, call ***mmw_motion_cube_access_close()*** before obtaining CFAR targets; otherwise, incorrect CFAR results will be obtained;
3. The interface for obtaining the CFAR target list may be called only once in the radar callback function for the same frame. Calls after the first will obtain incorrect CFAR results.

3.5 Obtaining the Frame-Transmission Timestamp

mmw_frame_start_timestamp()

Obtain the timestamp when the first Interval of the current frame begins transmission. This is useful for checking for DataCube access conflicts.

3.6 DataCube Data Access

This chapter describes commonly used functions for accessing DataCube. High-performance versions of these functions with the `__` prefix are also provided. Before calling an interface with the `__` prefix, ensure that ***mmw_motion_cube_access_open()*** and ***mmw_dsp_poweron()*** have been called.

For the ***interval_idx*** argument passed to DataCube-related interfaces, when Doppler FFT calculation is enabled through ***mmw_dop_fft_set()***, it is the Doppler index; when Doppler FFT calculation is disabled, it is the Interval index.

This chapter assumes that DataCube is a four-dimensional array: `DataCube(tx_num, rx_num, dfft_num, rfft_num)`;

If the 2D-FFT is executed, `fft_shift` has already been performed on the Doppler dimension. If the Doppler FFT length is `K`, `[0, K/2-1]` indicates negative velocity, `K/2` indicates zero velocity, and `[K/2+1, K]` indicates positive velocity.

For the range dimension, 0 indicates zero range, and the range FFT length indicates the maximum range.

3.6.1 Reading Receive Antenna Data at a Specified Position

mmw_fft_ants(complex16_cube *buffer, uint16_t interval_idx, uint16_t range_idx)

Obtain the data from all MIMO receive antennas at the specified `interval_idx` and `range_idx`:

`buffer = DataCube(:, :, interval_idx, range_idx)`

The output buffer is stored by iterating through all receive antennas first:

RS6130: `[tx0_rx0, tx0_rx1, tx0_rx2]`

RS624x: `[tx0_rx0, ..., tx0_rx3, tx1_rx0, ..., tx1_rx3]`

The interfaces derived from this interface include:

__mmw_fft_ants is the performance-optimized version of **mmw_fft_ants**. It provides faster data reading but no longer performs parameter checks.

__mmw_fft_abs_ants: reads data from all receive antennas at the specified position and calculates the magnitude;

__mmw_fft_logdB_ants: reads data from all receive antennas at the specified position and calculates the dB value (Q8);

3.6.2 Reading All Range FFT Values at a Specified Receive Antenna/Doppler Index

mmw_fft_range(complex16_cube *buffer, uint16_t read_num, uint8_t tx_id, uint8_t rx_id, uint16_t interval_idx)

Obtain a DataCube of the specified length along the range dimension, starting from 0, for the specified tx_id/rx_id/interval_idx:

```
buffer = DataCube(tx_id, rx_id, interval_idx, 0:read_num-1);
```

The interfaces derived from this interface include:

__mmw_fft_range is the performance-optimized version of **mmw_fft_range**. It provides faster data reading, no longer performs parameter checks, and allows configuration of the starting range index to be read.

__mmw_fft_abs_range: reads data for all range indices at the specified position and calculates the magnitude;

__mmw_fft_logdB_range: reads data for all range indices at the specified position and calculates the dB value (Q8);

3.6.3 Reading All Doppler FFT Values at a Specified Receive Antenna/Range Index

mmw_fft_doppler(complex16_cube *buffer, uint16_t read_num, uint8_t tx_id, uint8_t rx_id, uint16_t range_idx)

Obtain a DataCube of the specified length along the Doppler dimension, starting from 0, for the specified tx_id/rx_id/range_idx:

```
buffer = DataCube(tx_id, rx_idx, 0:read_num-1, range_idx);
```

__mmw_fft_doppler is the performance-optimized version of **mmw_fft_doppler**. It provides faster data reading, no longer performs parameter checks, and allows configuration of the starting Doppler index to be read.

__mmw_fft_abs_doppler: reads data for all Doppler indices at the specified position and calculates the magnitude;

__mmw_fft_logdB_doppler: reads data for all Doppler indices at the specified position and calculates the dB value (Q8).

3.6.4 DataCube Data Access Restrictions

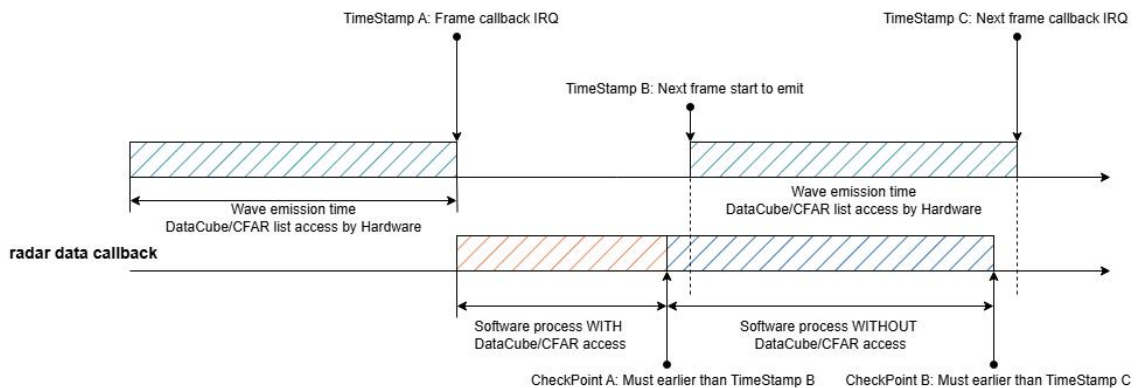


Figure 3-7 Checkpoints for Software Access to DataCube

CheckPoint A: When software and hardware access DataCube simultaneously, software may read incorrect DataCube data. Therefore, once waveform transmission begins, software must no longer read DataCube data; otherwise, incorrect values will be read.

CheckPoint B: If the callback for the previous frame has not exited when the interrupt for the next frame arrives, the interrupt for the next frame will not be serviced, and the next callback will be missed.

3.6.5 Configuring the Buffer for Range FFT Data from Multiple Intervals

mmw_2dfft_set_1dfft_buffer(complex16_cube *data_buff)

As shown in Figure 3-2, configure the range FFT data buffer for the specified Intervals. If the passed data_buff is nonzero, the hardware buffers the range FFT data to data_buff between the specified Intervals. If data_buff is 0, this function is disabled.

mmw_2dfft_set_1dfft_buffer can be called in a callback to specify a different Buffer for each frame, implementing ping-pong storage.

mmw_2dfft_obtain_1dfft_cfg(uint32_t range_start, uint32_t range_num, uint32_t start_intv, uint32_t end_intv, uint8_t tx_id, uint8_t rx_id)

The Interval range [start_intv, end_intv] and specified range-bin range [range_start, range_start + range_num] whose range FFT data is to be buffered can be configured.

The buffered range FFT data is arranged in memory in a manner similar to the following four-dimensional array:

```
data_cube[Interval_num][Tx_num][Rx_num][Range_FFT_num];
```

Range indices are filled first, followed by Interval indices, Rx indices, and Tx indices.

4 Q & A

4.1 How Can the Radar Waveform Be Switched in Real Time?

To switch the waveform, perform the following operations in order before the radar callback function ends:

1. Call `mmw_ctrl_stop()`
2. Reconfigure the waveform and baseband processing through the functions provided in Sections 3.2 and 3.3;
3. Call `mmw_ctrl_start()` to restart the radar and wait for the next interrupt.

Disclaimer and Copyright Notice

The information provided by our company in this document is accurate to the best of our knowledge, but we do not guarantee that the content is completely error-free. All liability for any damages arising from the use of this document (including but not limited to indirect, incidental, or special losses) or any improper use resulting from non-compliance with the instructions herein (including but not limited to over-voltage, over-frequency, or over-temperature operations) is explicitly disclaimed.

The content in this document, including URL references, may differ from the actual content due to product version upgrades or other reasons, and is subject to change without notice.

THIS DOCUMENT IS PROVIDED "AS IS" WITH NO WARRANTIES. NO WARRANTY IS PROVIDED TO THIS DOCUMENT FOR ITS MERCHANTABILITY, NON-INFRINGEMENT, FITNESS FOR ANY PARTICULAR PURPOSE, NOR DOES ANY WARRANTY OTHERWISE ARISING OUT OF ANY PROPOSAL, SPECIFICATION OR SAMPLE.

All liability, including liability for infringement of any patent rights, relating to use of information in this document is disclaimed. No licenses express or implied, by estoppel or otherwise, to any intellectual property rights are granted herein.

Copyright © 2026 Possumic Technology Co., Ltd. All rights reserved.